

# Ruta de auto-aprendizaje

Sección 2 — Repaso espaciado: matemáticas, algoritmos, implementación y recursos

Para dominar por cuenta propia todo lo que usa esta sección: de la curva de olvido a la cadena de Markov, con los ejemplos corriendo en el propio repo.

## 0. Mapa de la ruta

| Fase | Tema                                     | Tiempo orientativo | Resultado                                                                 |
|------|------------------------------------------|--------------------|---------------------------------------------------------------------------|
| 1    | Curva de olvido, Zipf y logística        | 1 semana           | Entender cada fórmula de engine/conocimiento.js                           |
| 2    | Cadenas de Markov                        | 1-2 semanas        | Derivar a mano $\pi$ , $E[\text{repasos}]$ y la tasa de engine/leitner.js |
| 3    | Algoritmos SRS: Leitner, SM-2, FSRS      | 1 semana           | Recalcular la tabla de ejemplo de SM-2 sin mirar el código                |
| 4    | Implementación y experimentos en el repo | 1-2 semanas        | Modificar la simulación y defender los resultados                         |

Regla de oro: cada concepto se aprende **tres veces** — en papel (la fórmula), en la consola (el número) y en el repo (el código que lo usa). Las tres vistas están indicadas en cada fase.

## Fase 1 — Herramientas matemáticas básicas

### 1.1 La curva de olvido (Ebbinghaus)

Modelo: la probabilidad de recordar algo  $\Delta$  días después del último repaso es  $R = e^{-(\Delta/S)}$ , donde  $S$  es la **estabilidad** (en días) de esa memoria. Propiedades que hay que interiorizar:  $R(0) = 1$ ;  $R(S) = e^{-1} \approx 0.37$  ( $S$  es «la vida media natural» del recuerdo, salvo constante); y para recordar al 90% hay que preguntar en  $\Delta \approx 0.105 \cdot S$  — de ahí que un SRS con retención objetivo del 90% programe intervalos de una décima de la estabilidad.

```
# En consola de Python:
import math
S = 6 # estabilidad de 6 días
for d in (0, 1, 3, 6, 12): print(d, round(math.exp(-d/S), 3))
# 0 1.0 | 1 0.846 | 3 0.607 | 6 0.368 | 12 0.135
```

Dónde vive en el repo: `retrievability()` en `engine/conocimiento.js` (con  $S = \max(\text{intervalo}, 0.5)$  y  $t$  recuperado del estado SM-2) y el alumno sintético de `simulacion/comparar.mjs` (con  $S$  que crece al acertar y se contrae al fallar).

### 1.2 La ley de Zipf y la escala zipf

En un corpus, la frecuencia de la palabra de rango  $r$  cae como  $\sim 1/r$ : pocas palabras acaparan casi todas las apariciones. La **escala zipf** normaliza el conteo para poder comparar entre corpus: **zipf** =  $\log_{10}(\text{conteo}/\text{total} \cdot 10^6)$  (apariciones por millón, en logaritmo). Referencias mentales: zipf  $\geq 5$  son palabras funcionales (und, der); zipf  $\approx 3$  es vocabulario medio; zipf  $< 2$ , palabras raras.

### 1.3 La función logística

$\sigma(x) = 1/(1+e^{-x})$  convierte una puntuación en probabilidad. El prior del modelo es  $P(\text{conocer}) = \sigma((\text{zipf} - z_0)/\tau)$  con  $z_0 = 3.0$  (el punto del 50%) y  $\tau = 0.7$  (cuán brusca es la transición). Ejercicio de 5 minutos: calcula  $P$  para «und» (zipf  $\approx 4.5$ ) y para un hapax (zipf  $\approx 1.5$ ) y comprueba contra la tabla de metricas-repaso.pdf.

## Fase 2 — Cadenas de Markov (el corazón matemático)

Una cadena de Markov es un proceso que salta entre estados donde el futuro solo depende del estado actual. Todo lo que usa esta sección cabe en cuatro ideas:

| Idea                            | Definición operativa                                                        | En Leitner                                                                |
|---------------------------------|-----------------------------------------------------------------------------|---------------------------------------------------------------------------|
| Matriz de transición $P$        | $P[i][j]$ = prob. de pasar de $i$ a $j$ ; filas suman 1                     | $P[i][i+1] = p$ (acierto), $P[i][1] = 1-p$ (fallo)                        |
| Distribución estacionaria $\pi$ | $\pi = \pi P$ ; a la larga, fracción de visitas a cada estado               | En qué caja «vive» una palabra a largo plazo                              |
| Tiempos de llegada              | $h_i = E[\text{pasos desde } i \text{ hasta un objetivo}]$ ; sistema lineal | Repasos/días esperados para llegar a la caja 5                            |
| Recompensas por estado          | Cada visita a $i$ «cuesta» $c_i$ ; coste medio = $\sum \pi_i \cdot c_i$     | Cada caja espera $l_i$ días $\rightarrow$ tasa = $1/\sum \pi_i \cdot l_i$ |

### 2.1 Ejemplo resuelto: Leitner con $p = 0.9$

$E[\text{repasos hasta la caja 5}]$  es la esperanza de la primera racha de 4 aciertos seguidos:  $E = 1/p + 1/p^2 + 1/p^3 + 1/p^4$ . Con  $p = 0.9$ :  $1.111 + 1.235 + 1.372 + 1.524 \approx 5.24$  repasos. Derivación (hazla en papel):  $h_i = 1 + p \cdot h_{i+1} + (1-p) \cdot h_1$  con  $h_5 = 0$ ; sustituye hacia atrás y despeja  $h_1$ . Los días esperados usan la misma recurrencia cambiando el «1» por el intervalo  $l_i$  de cada caja ( $\approx 18.1$  días con  $p = 0.9$ ).

```
# Compruébalo contra el motor real (consola de Node en la raíz del repo):
node -e "import('./src/engine/leitner.js').then(L => {
  console.log(L.repasosHastaUltimaCaja(0.9)); // 5.2415...
  console.log(L.diasHastaUltimaCaja(0.9)); // 18.09...
  console.log(L.distribucionEstacionaria(L.matrizTransicion(0.9)));
})"
```

### 2.2 Qué estudiar y dónde

| Recurso                                                                                                     | Qué aporta                                                                                                                        |
|-------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| Grinstead & Snell, «Introduction to Probability», cap. 11 (PDF libre en la web de Dartmouth)                | El capítulo canónico de cadenas de Markov: regulares, absorbentes, tiempos de absorción — exactamente la matemática de leitner.js |
| Kemeny & Snell, «Finite Markov Chains»                                                                      | Referencia clásica si quieres más profundidad (matriz fundamental $N = (I-Q)^{-1}$ )                                              |
| Vídeos: 3Blue1Brown (probabilidad) y cualquier serie de «Markov chains» (p. ej. Normalized Nerd en YouTube) | Intuición visual de estados y estacionarias                                                                                       |
| setosa.io/ev/markov-chains                                                                                  | Visualización interactiva para jugar con matrices                                                                                 |

## Fase 3 — Los algoritmos SRS

### 3.1 Leitner (1972): el algoritmo de las cajas

5 cajas con cadencias [1, 2, 4, 8, 16] días; acierto sube, fallo devuelve a la caja 1. Virtud: trivial de explicar y de implementar con fichas físicas. Defecto estructural (cuantificado en [metricas-repaso.pdf](#)): el intervalo tiene **techo**, así que un mazo maduro cuesta  $N/16$  repasos diarios para siempre.

### 3.2 SM-2 (Wozniak, 1987): intervalos que crecen con el historial

Cada ítem lleva (n, EF, I). Tras una respuesta de calidad  $q \in 0..5$ : si  $q < 3$ ,  $n = 0$  e  $I = 0$  (reaprender); si  $q \geq 3$ ,  $n$  aumenta e  $I(1) = 1$ ,  $I(2) = 6$ ,  $I(n) = \text{round}(I(n-1) \cdot EF)$ . El factor  $EF' = \max(1.3, EF + 0.1 - (5-q) \cdot (0.08 + (5-q) \cdot 0.02))$  baja con «Difícil» ( $q=3$ ,  $-0.14$ ), no cambia con «Bien» ( $q=4$ ) y sube con «Fácil» ( $q=5$ ,  $+0.10$ ). Ejemplo trabajado con EF constante 2.5 (todo «Bien»):

| Repaso | q        | n tras | EF tras | Intervalo (días) | Vence (día acumulado) |
|--------|----------|--------|---------|------------------|-----------------------|
| 1      | 4 (Bien) | 1      | 2.50    | 1                | 1                     |
| 2      | 4 (Bien) | 2      | 2.50    | 6                | 7                     |
| 3      | 4 (Bien) | 3      | 2.50    | 15               | 22                    |
| 4      | 4 (Bien) | 4      | 2.50    | 38               | 60                    |
| 5      | 4 (Bien) | 5      | 2.50    | 95               | 155                   |
| 6      | 4 (Bien) | 6      | 2.50    | 238              | 393                   |

Seis aciertos llevan la palabra a un intervalo de ~94 días: el crecimiento geométrico es lo que abarata SM-2 frente a Leitner. Reproduce esta tabla a mano y luego con el motor:

```
node -e "import('./src/engine/srs.js').then(S => {
  let e; const T = '2026-01-01T00:00:00.000Z';
  for (let i = 0; i < 6; i++) { e = S.calificar(e, 4, T); console.log(e.reps, e.ef, e.intervalo); }
})"
```

### 3.3 Más allá: half-life regression y FSRS

La frontera actual ajusta la estabilidad por regresión sobre datos reales: **Half-Life Regression** (Settles & Meeder, ACL 2016 — el paper de Duolingo, muy legible) y **FSRS** (Free Spaced Repetition Scheduler, el algoritmo moderno de Anki: estado (Dificultad, Estabilidad, Retrievability) con actualizaciones aprendidas de millones de repasos; su wiki en GitHub, [open-spaced-repetition/fsrs4anki](#), trae toda la matemática). Son la extensión natural de esta sección si se quiere ir más lejos.

## Fase 4 — La implementación de este repo, por dentro

### 4.1 Insights de diseño que conviene entender

| Insight                                                              | Por qué importa                                                                     |
|----------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| Motores puros con tiempo inyectado (ahora como parámetro ISO)        | El mismo código corre en la UI, en los tests y en la simulación; sin mocks de reloj |
| El estado SRS vive dentro de la entrada de la bolsa                  | Una sola fuente de verdad en localStorage; agregar una palabra existente no lo pisa |
| El último repaso no se guarda: se deriva (vencimiento – intervalo)   | El estado mínimo suficiente; menos campos que puedan desincronizarse                |
| Separar planificador (cuándo preguntar) de memoria (qué se recuerda) | Es la estructura de la simulación y de toda la literatura SRS                       |

| Insight                                               | Por qué importa                                                                                                                                  |
|-------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| Contabilidad fiel a cada método al comparar           | Un fallo SM-2 cuesta 2 presentaciones (recicla en sesión); en Leitner 1 — compararlos con la misma contabilidad falsearía el coste               |
| Calibrar el alumno sintético antes de leer resultados | Con S0 de horas la precisión cae al 25% y la comparación no informa; con precisión 80-90% (como los SRS reales) los resultados son transferibles |

## 4.2 Paquetería y herramientas

| Herramienta             | Rol en la sección                                                                      |
|-------------------------|----------------------------------------------------------------------------------------|
| Vitest 4                | 49 tests del engine (npm test); la especificación ejecutable de cada fórmula           |
| Node ≥ 20 (ESM)         | Corre la simulación directamente sobre src/engine/ (npm run simular)                   |
| mulberry32              | RNG determinista de 32 bits: misma semilla → misma simulación                          |
| ReportLab               | Genera estos PDF (docs/generar_*.py), tablas y gráficas nativas                        |
| Python 3 + PYTHONUTF8=1 | pipeline/frecuencias.py y los generadores (regla: ensure_ascii=False para los umlauts) |
| localStorage            | Persistencia sin backend: bolsa.v1, conocidas.v1, repasoPrevio.v1                      |

## 4.3 Experimentos propuestos (en orden de dificultad)

1) Cambia la semilla y el horizonte de simulación/comparar.mjs y verifica que las conclusiones no dependen del azar. 2) Añade una caja 6 (intervalo 32) a INTERVALOS\_CAJA y mide cuánto trabajo ahorra: ¿se acerca a SM-2? Predícelo antes con la fórmula de la tasa ( $1/\Sigma \pi \cdot I$ ). 3) Haz que el alumno responda «Fácil» cuando  $R > 0.95$  y «Difícil» cuando  $R < 0.6$ , y observa cómo EF separa palabras fáciles de difíciles. 4) Implementa la retención objetivo: programa el siguiente repaso en  $\Delta = -S \cdot \ln(0.9)$  en lugar de usar SM-2, y compárala con ambos (es la idea central de FSRS). 5) Deriva la matriz fundamental  $N = (I - Q)^{-1}$  de la cadena absorbente (caja 5 absorbente) y comprueba que  $N \cdot 1$  reproduce repasosHastaUltimaCaja(p).

## 5. Recursos externos (lista corta y buena)

| Recurso                                                                                                                           | Tipo                | Para qué                                           |
|-----------------------------------------------------------------------------------------------------------------------------------|---------------------|----------------------------------------------------|
| Wozniak — descripción original de SM-2 (supermemo.guru / super-memory.com, «Application of a computer to improve the results...») | Fuente primaria     | El algoritmo tal como se publicó                   |
| Manual de Anki (docs.ankiweb.net), sección «What algorithm»                                                                       | Documentación       | Cómo un producto real adapta SM-2/FSRS             |
| Settles & Meeder (2016), «A Trainable Spaced Repetition Model for Language Learning» (ACL)                                        | Paper               | Half-life regression con datos de Duolingo         |
| open-spaced-repetition/fsrs4anki (GitHub, wiki «The Algorithm»)                                                                   | Código + docs       | FSRS: el estado del arte, con fórmulas             |
| Murre & Dros (2015), «Replication and Analysis of Ebbinghaus' Forgetting Curve» (PLOS ONE)                                        | Paper               | La curva de olvido con datos modernos              |
| Grinstead & Snell, cap. 11 (Dartmouth, PDF libre)                                                                                 | Libro               | Cadenas de Markov completas con ejercicios         |
| gwnet.net/spaced-repetition                                                                                                       | Ensayo/recopilación | Panorama y bibliografía comentada de todo el campo |

## 6. Lista de verificación de dominio

Puedes dar la sección por dominada cuando, sin mirar el código: (a) escribes la recurrencia de SM-2 y explicas por qué «Bien» no mueve EF; (b) derivas  $E = \sum p^j$  para la racha de aciertos de Leitner; (c) explicas por qué la tasa estacionaria es  $1/\sum \pi \cdot l$  y la calculas para  $p = 0.9$ ; (d) justificas los cuatro casos de  $P(\text{conocer})$  del modelo de conocimiento; (e) predices el efecto de añadir una caja a Leitner antes de simularlo — y luego lo confirmas con `npm run simular`.